Module 5: Delegates

GOPAS

Overview

- Creating Delegate
- Instantiate Delegate
- Invocation List
- Using Delegates for CallBack
- Events
- Event Guidelines

Delegates Introduction

- Delegates are like C++ function pointers but are type safe
- A delegate is a type that represents references to methods with a particular parameter list and return type.
- You can associate its instance with any method with a compatible signature and return type
- You can invoke (or call) the method through the delegate instance

- Delegates can be used to
  - define callback methods.
  - define events
  - asynchronous programming
  - anonymous methods
  - lambda expressions

Using Delegates

```
0 references
void MyMethod(string s)
{
}
```

- Define delegate type

```
public delegate void MyDelegate(string message);
```

- Instantiate delegate

```
// Instantiate the delegate.
MyDelegate d = new MyDelegate(MyMethod);
```

- Call delegate

```
// Call method using the delegate
d.Invoke("Hello World");

// Call method directly
MyMethod("Hello World");
```

Using Delegates

```
0 references
void MyMethod(string s)
{
}
```

- Instantiate delegate

```
MyDelegate d = new MyDelegate(MyMethod);
```

```
MyDelegate d = MyMethod;
```

- Call delegate

```
d.Invoke("Hello World");
```

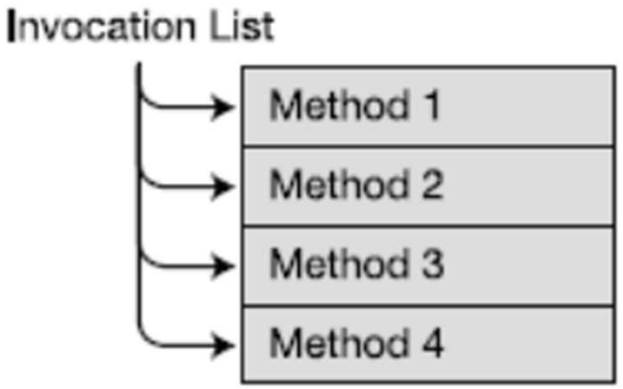
```
d("Hello World");
```

GOPAS

Invocation List

- Delegate can have more than one method in their invocation list

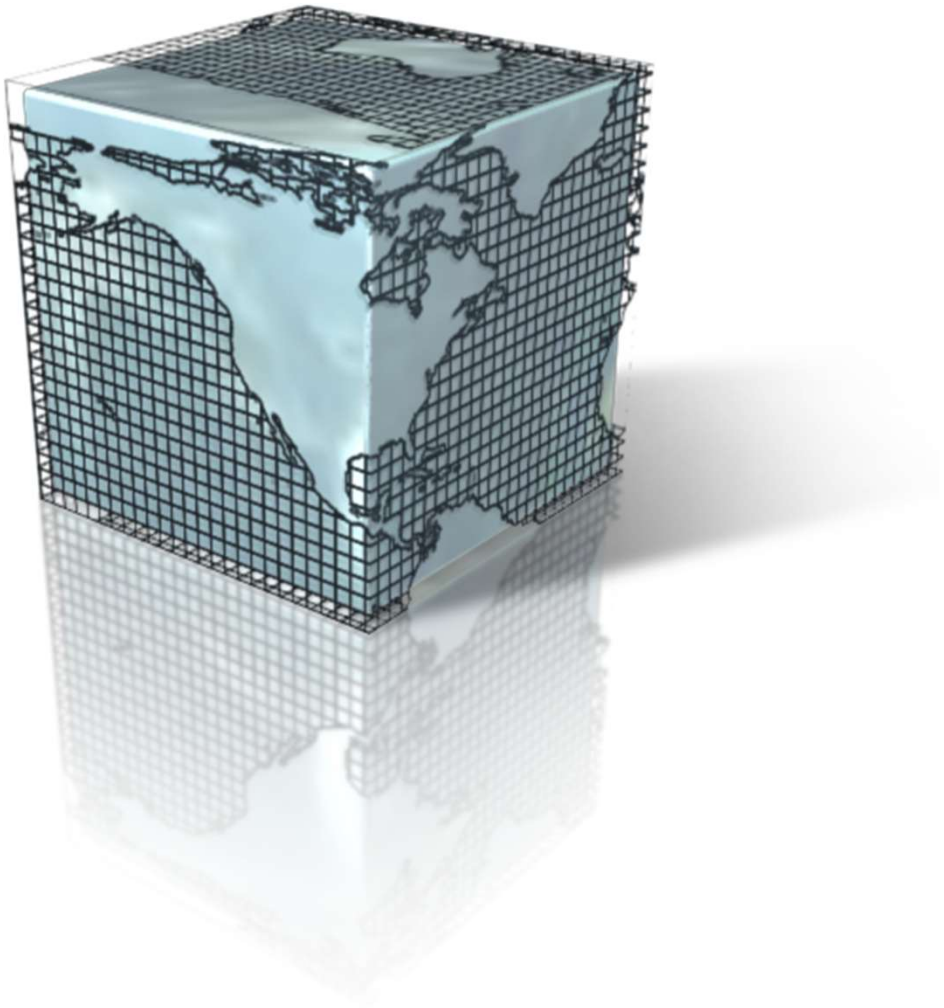
```
// Combining delegates.
MyDelegate.Combine(d1, d2);
```



- or

```
// Combining delegates.
d1 + d2;
```

Lab 05.1 - Using Delegates



Example:

1. Creating delegate
2. Combining delegates

GOPAS

Using Delegates for CallBack

- *Here's my number* (delegate),
- *So if something happen* (event),
- *Call me, maybe ?* (callback)

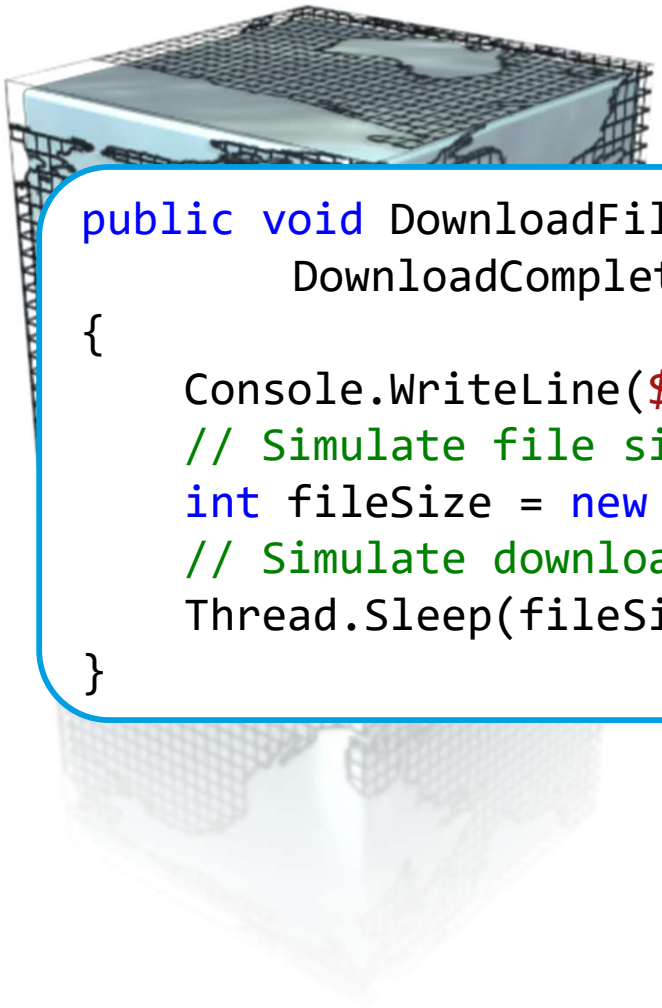
Lab 05.2 - Using Delegates for CallBack

Example:

1. CallBack and Delegates

GOPAS

Exercise – FileDownloader CallBack



- Create class FileDownloader with method

```
public void DownloadFile(string fileUrl,
        DownloadCompletedCallback callback)
{
    Console.WriteLine($"Starting download from {fileUrl}...");
    // Simulate file size between 1 KB and 10 KB
    int fileSize = new Random().Next(1_000, 10_000);
    // Simulate download delay
    Thread.Sleep(fileSize);
}
```

- Add method to Program class

```
void OnDownloadCompleted(int fileSize)
```

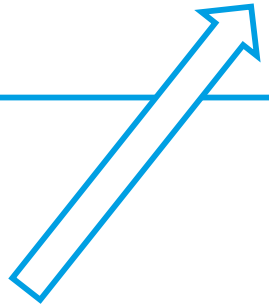
- Use callback to call this function on file download complete

GOPAS

Keyword event

- Declaring event as a variable typed delegate with event. keyword

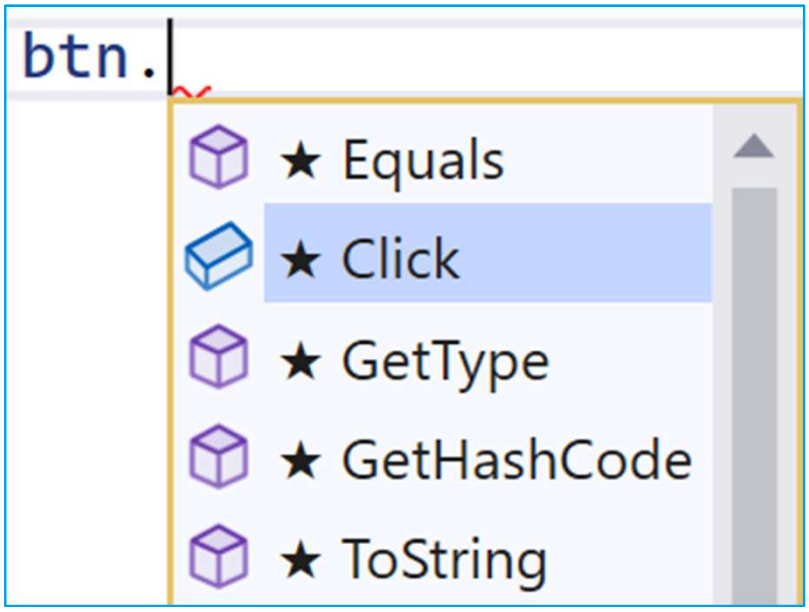
```
public delegate void ClickEventHandler();
class Button
{
    public event ClickEventHandler Click;
}
```



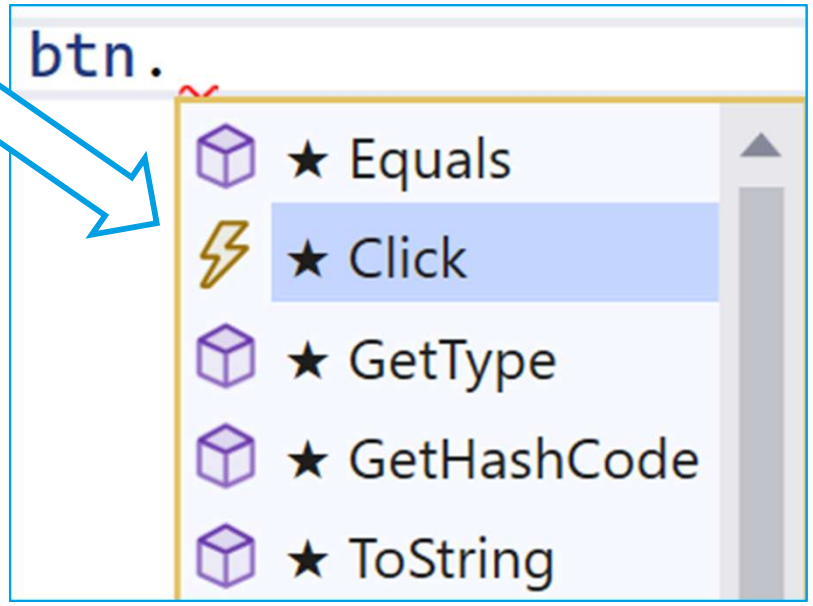
Event vs. Delegate

- Changed Icon in Visual Studio

Delegate



Event



Event vs. Delegate

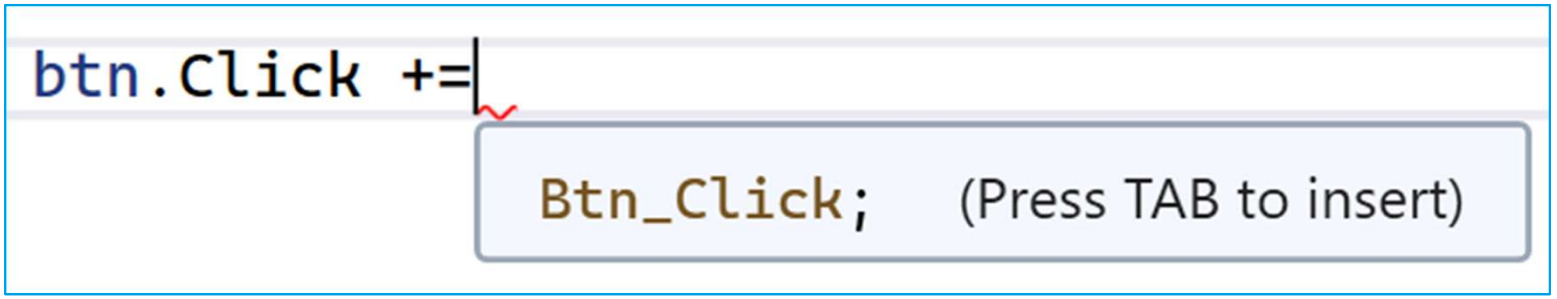
- Registering event handler using Combine() method

Delegate

```
btn.Clicked = MyMethod;
```

Event

```
btn.Clicked += MyMethod;
```



GOPAS

Defining Events

Publisher

1) Defining an event

```
public event ClickEventHandler Click;
```

2) Notifying subscribers to an event

```
if (this.Click != null) {
        this.Click();
}
```

Subscriber

1) Subscribing to an event

```
btn.Click += new ClickEventHandler(btn_Click);
```

Callback Pattern

- Defined typically on function level
- Callbacks are typically initiated by the function that receives them in parameter

```
public delegate void ProgressDelegate(int progress);

public class DataProcessor
{
    public void ProcessData(ProgressDelegate callback)
    {
        if (callback != null)
        {
            callback(indicator);
        }
    }
}
```

Events Pattern

- Defined typically on object level
- Events are initiated by an external source

```
public delegate void ProgressDelegate(int progress);

public class DataProcessor
{
    public ProgressDelegate Progress;
    public void ProcessData()
    {
        if (Progress != null)
        {
            Progress(indicator);
        }
    }
}
```

Lab 05.3 - Simple Event

Copy Lab 5.2

Exercise:

1. From CallBack to an Event

GOPAS

EventHandler Delegate

- Built-in delegate for implementing events.

```
public delegate void EventHandler(Object sender, EventArgs e);
```

- Two parameters:
  - `object source` - parameter indicating the source of the event
  - `EventArgs e` - or derived from the **EventArgs** class

Lab 05.4 - Using Delegates for Events

Example:

1. Events and Delegates
2. Custom EventArgs argument

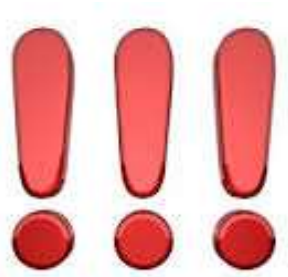
```
class ListWithChangedEvent<T> : List<T>
```

GOPAS

Events in Derived Classes

- Events are a special type of delegate that can only be invoked from within the class that declared them

```
public event EventHandler WorkDone;
```



- To enable the derived class to invoke base class events create a protected invoking method in the base class that wraps the event

```
protected virtual void OnWorkDone(EventArgs e)
```

Event Accessors

- Public delegate

```
public event EventHandler Click;
```

- Event accessor

```
private EventHandler clickHandler;
public event EventHandler Click
{
        add { clickHandler += value; }
        remove { clickHandler -= value; }
}
```

Note:

- *Custom behavior for registering and unregistering an event*

Lab 05.5 - Event Accessors and Derived Classes

Example:

1. Standard Event
2. Event Accessors
3. Derived Classes

GOPAS

Exercise - Events (Dice Game)

- Create class Dice with a
  - method Roll()
  - readonly Property Value
  - event Changed
- Create 6 instances of class Dice
- If any Dice is rolled let the event Changed to be thrown

GOPAS

Review

- Creating Delegate
- Instantiate Delegate
- Invocation List
- Using Delegates for CallBack
- Events
- Event Guidelines